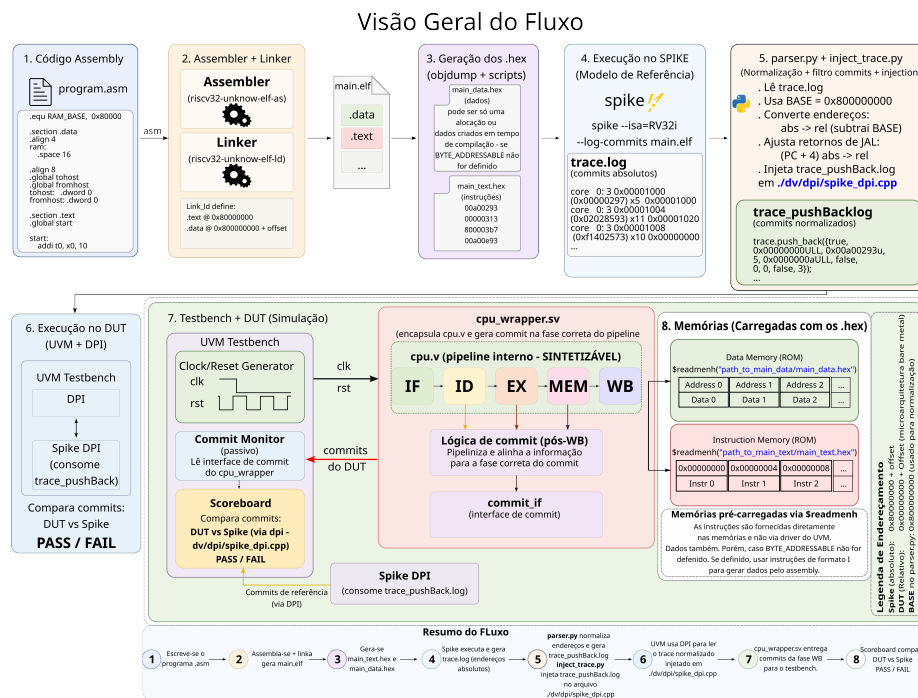


Fluxo de Verificação

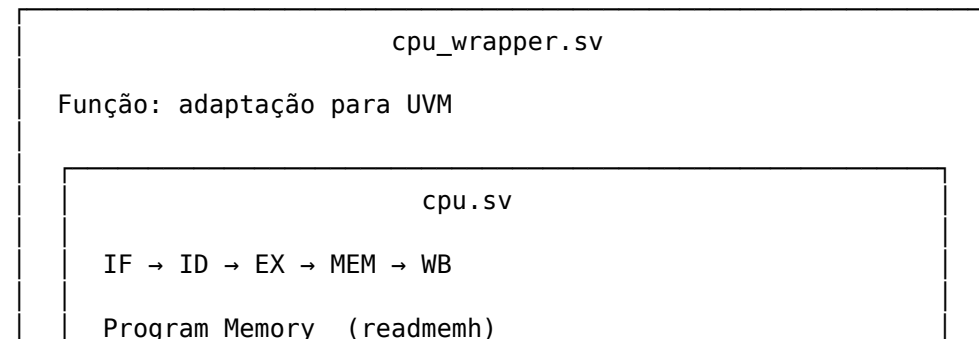
Este projeto testa microarquitecturas RISC-V através de UVM / DPI.

É obrigatório a implementação da instrução LUI para carregamento do endereço inicial 0x80000000 que o SPIKE utiliza.

Encapsule seu projeto com um WRAPPER como o dado a seguir para sincronização e exposição de sinais necessários aos commits de verificação entre DUT e SPIKE.



Papel do cpu_wrapper.sv



Data Memory (readmemh)
Register File

sinais internos do DUT

pc_WB_Arch rd_WB writeBack_WB
↓ ↓ ↓

Commit Adapter Logic

valid = regFileWE_WB && rd_WB!=0

pc = pc_WB_Arch - 4
rd_addr = rd_WB
rd_data = writeBack_WB

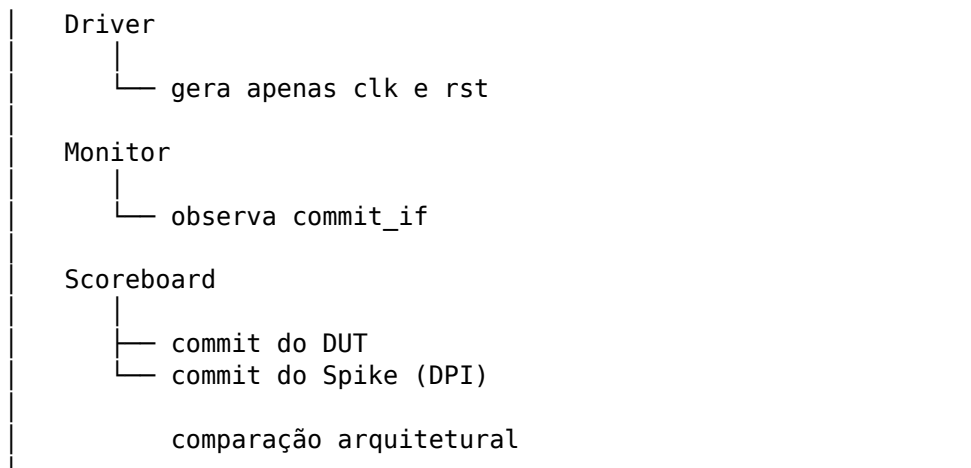
Pipeline auxiliar de instrução

inst_IF
↓
inst_ID_commit
↓
inst_EX_commit
↓
inst_MEM_commit
↓
commit_instr

(não existe originalmente em cpu.sv)

↓
commit_if interface
↓

UVM



Por exemplo, o pipeline auxiliar para gerar o commit da instrução na fase correta(WB) junto com os outros sinais:

cpu.sv

```

IF → ID → EX → MEM → WB
|
└─ instrução disponível apenas nos estágios iniciais

```

cpu_wrapper.sv cria pipeline auxiliar para sincronizar o estágio WB da instrução com

```

inst_IF
|
▼
inst_ID_commit
|
▼
inst_EX_commit
|
▼
inst_MEM_commit
|
▼
commit_instr

```

Objetivo:
fazer a instrução chegar ao mesmo ciclo em que o resultado arquitetural aparece em WB.

⚠ OBSERVAÇÃO

cpu_wrapper.sv NÃO faz parte da microarquitetura.

Ele existe apenas para integração com o ambiente UVM, permitindo:

1. exposição do estado arquitetural necessário ao commit;
2. alinhamento temporal dos sinais até WB;
3. criação de pipelines auxiliares de verificação;
4. adaptação de reset e interfaces.

A síntese do processador deve ser realizada diretamente sobre **cpu.sv**.

Assim, alterações realizadas no wrapper, mesmo que utilizando estruturas sintetizáveis, não afetam a lógica sintetizável do processador.

📁 Estrutura esperada de diretórios

```
project/
├── assembly
│   ├── link.ld
│   │   Entrada - Script do linker. Define organização da memória (text, data, stack)
│   ├── program.asm
│   │   Entrada - Programa em assembly RISC-V a ser compilado e executado
│   ├── prog_bolhas_linker_inicializa_mem.asm
│   │   Entrada - Programa assembly com bolhas (NOPs) e inicialização de memória
│   ├── prog_bolhas_linker_inicializa_mem_JAL_JALR.asm
│   │   Entrada - Variante do programa usando JAL/JALR para controle de fluxo
│   ├── main.o
│   │   Saída - Código objeto gerado pelo assembler (binário intermediário)
│   ├── main.elf
│   │   Saída - Executável final RISC-V gerado pelo linker (entrada do Spike/UVM)
│   ├── main_text.hex
│   │   Saída - Memória de instruções (.text) em formato hex (1 instrução por linha)
│   ├── main_data.hex
│   │   Saída - Memória de dados (.data) em formato hex (inicialização da RAM)
│   └── trace.log
│       Saída - Log bruto do Spike com commits de execução
```

- └─ trace_pushBack.log
 Saída - Trace convertido para formato C++ (trace.push_back) para UVM/DPI
- docs
 - └─ LICENSE
 - └─ README.md
- dv
 - └─ dpi
 - └─ spike_dpi.cc
 Entrada - Modelo DPI em C++ (mock do Spike ou integração com trace)
 - └─ spike_dpi_wrapper.sv
 Entrada - Wrapper SystemVerilog que conecta DUT ao DPI-C
 - └─ env
 - └─ ref_env_pkg.sv
 Entrada - Ambiente UVM (environment package com agents, config e estrutura)
 - └─ if
 - └─ commit_if.sv
 Entrada - Interface SystemVerilog para sinais de commit entre DUT e UVM
 - └─ include
 - └─ riscv_defs.svh
 Entrada - Defines globais do RISC-V (opcode, width, macros, constantes)
 - └─ scoreboard
 - └─ ref_scoreboard.sv
 Entrada - Scoreboard UVM que compara DUT vs referência (Spike/DPI)
- filelist.f
 Entrada - Lista de arquivos RTL para o simulador (Xcelium/Quarta/etc)
- hw
 - └─ rtl - arquivos que implementam seu projeto
 - └─ alu.v
 Entrada - Unidade Aritmética e Lógica (ALU do processador)
 - └─ aludec.v
 Entrada - Decodificador de controle da ALU
 - └─ branchControl.v
 Entrada - Lógica de controle de desvios (branch/jump)
 - └─ control.v
 Entrada - Unidade de controle principal (gera sinais de controle)
 - └─ cpu.v

	Entrada - Top-level do core RISC-V (CPU principal)
—	cpu_TB.v
	Entrada - Testbench básico da CPU (sem UVM)
—	cpu_wrapper.v
	Entrada - Wrapper do CPU para integração com ambiente UVM/DPI
—	cpu_wrapper.sv
	Entrada - Versão SystemVerilog do wrapper (interface UVM/DPI)
—	cpu_wrapper_TB.v
	Entrada - Testbench do wrapper da CPU
—	datapath.v
	Entrada - Caminho de dados (datapath do processador)
—	extend.v
	Entrada - Extensor de imediato (sign/zero extend)
—	maindec.v
	Entrada - Decodificador principal de instruções
—	Memory.v
	Entrada - Memória principal (instruções/dados)
—	MemoryByteAddressable
	Entrada - Variante de memória byte-addressable (modelo alternativo)
	— byteEnableDecoder.v
	— memByteAddressable32WF.v
	— memory_write_first.v
	— memReadManager.v
	— memTopo32LittleEndian.v
—	mux.v
	Entrada - Multiplexador básico
—	mux4.v
	Entrada - Multiplexador 4:1
—	pc.v
	Entrada - Program Counter (PC)
—	register.v
	Entrada - Registrador simples (flip-flop base)

```

|       |— registerFile.v
|       |   Entrada - Banco de registradores RISC-V
|— images
|   Imagens de documentação
|
|— Makefile
|   Entrada - Automação do fluxo (build, spike, trace, parser, UVM)
|— README.md
|— sim
|   |— riscv_tb_top.sv
|   |   Entrada - Testbench top-level do ambiente UVM/SystemVerilog
|   |
|— tools
|   |— regdiff.py
|   |   Entrada - Ferramenta Python para comparação de registradores (DUT vs referên
|   |
|   |— inject_trace.py
|   |   Entrada - Script para injeção/geração de trace para UVM/DPI
|   |
|   |— parser.py
|   |   Entrada - Parser do trace do Spike → formato trace.push_back() UVM

```

Fluxo de Execução com o Make:

O Makefile automatiza:

- montagem/compilação do programa RISC-V (.asm -> .elf)
- geração do arquivo .hex (para uso com readmenh na inicialização de memória)
- execução no Spike
- geração do trace.log
- conversão automática para trace_pushBack.log (formato aceito pelo UVM scoreboard)
- execução da simulação UVM com Xcelium (xrun)

IMPORTANTE: selecionar o programa assembly

Antes de rodar qualquer alvo, altere no Makefile a variável:

```
SRC = $(ASM_DIR)/program_lui_RAM_BASE_0x800000.asm
```

Troque `program_lui_RAM_BASE_0x80000.asm` pelo nome do seu arquivo assembly.

Exemplo:

```
SRC = $(ASM_DIR)/sum_loop.asm
```

ou

```
SRC = $(ASM_DIR)/branch_test.asm
```

Comandos disponíveis

⚙ 1. Compilar ELF

```
make
```

Gera:

```
assembly/main.o  
assembly/main.elf
```

⚙ 2. Gerar HEX

```
make hex
```

Gera:

```
assembly/main.hex
```

Formato:

- Intel HEX
 - compatível com `$readmemh`
-

⚙ 3. Gerar trace do Spike + parser UVM + inject_trace no cpp

```
make trace
```

Executa:

```
spike --isa=RV64IM_ZICSR_ZIFENCEI \  
      --log-commits \  
      /opt/riscv/riscv64-unknown-elf/bin/pk \  
      assembly/main.elf
```


Depois roda:

```
python3.12 ./tools/parser.py
```

Saídas:

```
assembly/trace.log
```

```
assembly/trace_pushBack.log
```

e o inject_trace.py:

```
python3.12 ./tools/inject_trace.py
```

❏ 4. Fazer todo processo de compilação / geração de trace_pushBack.log / inserção em dv/dpi/spike_dpi.cc

```
make run
```

Executa:

1. build ELF
 2. gera HEX
 3. roda Spike
 4. roda parser
 5. injeta no spike_dpi.cc Pronto para simulação UVM
-

❏ 5. Rodar simulação UVM

Em sua cpu.v e implementação das memórias da microarquitetura, deixe o arquivo de inicialização de memória de programa e/ou dados parametrizada, como neste projeto, para facilitar a troca de programas/dados.

No arquivo **hw/rtl/cpu_wrapper_tb.v** modifique

```
parameter INIT_FILE_PROG = "../../assembly/main_text.hex";  
//!Arquivo com o código de máquina em formato hexadecimal
```

se necessário. Seguindo o fluxo contido nesse Makefile, não é necessário modificar.

O Makefile também gera o arquivo de inicialização da memória RAM (Memória de Dados). No entanto, lembre-se que, se a microarquitetura estiver implementando uma memória Byte Addressable, essa inicialização não irá funcionar e você terá que inicializar a memória através de instruções imediatas em seu programa assembly. Neste

caso não é necessário fazer modificações. A inicialização da memória de dados simplesmente será ignorada.

⚠ **Observação:** É possível fazer a memória Byte Addressable inicializar corretamente cada bloco de 8 bits, mas você terá que mudar a implementação da mesma (adicionando readmenh para arquivos distintos), o fluxo para gerar 4 bancos de inicialização distintos de main_data.hex (cada um contendo o byte pertinente do dado de 64 bits) e os parâmetros de inicialização no testbench/topo para que cada bloco da ByteAddressable receba o byte correto da palavra na inicialização. E lembre-se que o RISC-V é little-endian.

```
make sim
```

Executa:

```
xrun
```

com:

- UVM
 - DPI
 - filelist.f
 - smoke_test
-

🔧 6. Rodar simulação com GUI

```
make gui
```

Abre o Xcelium GUI com waveform habilitada.

🗑 7. Limpar arquivos

Sempre rode ***make clean*** antes de um novo teste.

```
make clean
```

Remove:

```
assembly/*.elf
assembly/*.o
assembly/*.hex
assembly/*.log
xrun.history
xcelium.d
INCA_libs
*.shm
```

```
*.vcd  
*.vpd  
worklib
```

Fluxo recomendado

Para um teste completo:

```
make clean && make run && make sim
```

ou para debug gráfico:

```
make clean && make run && make gui
```

Requisitos

1. Toolchain:

```
/opt/riscv/bin/riscv64-unknown-elf-*
```

A RISC-V GNU Toolchain é o conjunto de ferramentas baseado no projeto GNU utilizado para desenvolver software para processadores RISC-V. Ela fornece compiladores, montadores, ligadores e utilitários responsáveis por transformar código-fonte em binários executáveis compatíveis com a ISA RISC-V.

Neste projeto, a toolchain é utilizada para montar, ligar, inspecionar e converter programas assembly utilizados nos testes de verificação.

As principais ferramentas utilizadas são:

Principais Ferramentas da RISC-V GNU Toolchain

- riscv64-unknown-elf-as
Montador (assembler) responsável por converter código assembly RISC-V em arquivos objeto (.o).
- riscv64-unknown-elf-gcc
Driver principal da toolchain GNU. Neste projeto é utilizado para realizar a etapa de linkedição dos arquivos objeto, gerando o executável ELF final. O GCC invoca internamente o linker e demais ferramentas necessárias, simplificando o fluxo de compilação e garantindo compatibilidade com a configuração da toolchain.

- `riscv64-unknown-elf-ld`
Linker GNU utilizado pelo GCC durante a etapa de linkedição. É responsável por posicionar as seções de código e dados na memória de acordo com o linker script (`link.ld`).
- `riscv64-unknown-elf-objdump`
Utilitário de inspeção utilizado para visualizar instruções, seções, endereços e conteúdo do executável ELF.
- `riscv64-unknown-elf-objcopy`
Utilitário utilizado para converter o executável ELF em outros formatos, como arquivos HEX utilizados para inicialização das memórias do DUT.

Repositório oficial:

<https://github.com/riscv-collab/riscv-gnu-toolchain>

⚡ 2. Spike - simulador de referência oficial da ISA RISC-V, mantido pela comunidade RISC-V International - golden reference model

`spike`

O Spike é o simulador de referência oficial da ISA RISC-V, mantido pela comunidade RISC-V International. Seu objetivo principal é fornecer um modelo funcional (golden reference model) do comportamento arquitetural definido pelas especificações da ISA.

Diferentemente de simuladores RTL, o Spike não modela detalhes de microarquitetura, como pipeline, hazards, forwarding, caches ou temporização. Ele executa instruções de forma funcional, produzindo o estado arquitetural esperado após cada instrução.

Neste projeto, o Spike é utilizado como modelo de referência para gerar o trace arquitetural de execução do programa assembly. Esse trace é posteriormente convertido para o formato utilizado pelo ambiente UVM e comparado com os commits produzidos pelo DUT (Design Under Test).

O fluxo adotado utiliza a opção `--log-commits`, permitindo registrar, para cada instrução que altera o estado arquitetural, informações como:

- Program Counter (PC);
- Instrução executada;
- Registrador de destino (rd);
- Valor escrito no registrador.

Essas informações são utilizadas pelo scoreboard UVM para verificar a equivalência funcional entre a implementação RTL e o modelo de referência.

Importante: o Spike modela um sistema de memória unificado (Von Neumann), enquanto a microarquitetura deste projeto utiliza memórias de instrução e dados separadas (Harvard). Por essa razão, foi necessário realizar uma normalização dos endereços no parser de trace para compatibilizar os commits gerados pelo Spike com os endereços utilizados internamente pelo DUT.

Repositório:

<https://github.com/riscv-software-src/riscv-isa-sim>

3. Python:

python3.12

4. Xcelium:

xrun

Verificação final da Infraestrutura

Todos os comandos devem existir:

```
which riscv64-unknown-elf-gcc
which riscv64-unknown-elf-objdump
which spike
```

Saída esperada:

```
/opt/riscv/bin/riscv64-unknown-elf-gcc
/opt/riscv/bin/riscv64-unknown-elf-objdump
/opt/riscv/bin/spike
```

Observação

Este projeto assume os caminhos:

```
/opt/riscv/bin/  
/opt/riscv/riscv64-unknown-elf/bin/pk
```

Se instalar em outro local, atualize o Makefile.

⚙️ Seleção correta da ISA no Makefile e em tools/parser.py

O projeto utiliza a toolchain RISC-V para montar, linkar e executar os programas assembly. É extremamente importante que a ISA configurada no Makefile seja compatível com:

1. a microarquitetura implementada no DUT;
2. as instruções utilizadas no programa assembly;
3. o Spike;
4. o linker flags (-march e -mabi).
5. Configuração da Toolchain

No Makefile, ajuste os caminhos da toolchain:

⚙️ 1. TOOLCHAIN RISC-V

```
AS = /opt/riscv/bin/riscv64-unknown-elf-as  
LD = /opt/riscv/bin/riscv64-unknown-elf-ld  
CC = /opt/riscv/bin/riscv64-unknown-elf-gcc  
OBJCOPY = /opt/riscv/bin/riscv64-unknown-elf-objcopy  
OBJDUMP = /opt/riscv/bin/riscv64-unknown-elf-objdump  
SPIKE = /opt/riscv/bin/spike
```

⚙️ 2. Configuração da ISA (-march)

A ISA utilizada na compilação é definida principalmente pela flag:

LDFLAGS = -march=rv32i -mabi=ilp32 -nostdlib -T \$(LDFILE)

Caso deseje utilizar extensões diferentes da ISA, altere o parâmetro **-march**.

Principais variantes suportadas

1. Configuração RV32I

-march=rv32i

ISA base inteira de 32 bits.

Inclui apenas:

1. operações inteiras
2. load/store
3. branches
4. jumps
5. instruções lógicas/aritméticas básicas

Exemplos:

1. add
2. sub
3. lw
4. sw
5. beq
6. jal

Não possui:

1. multiplicação/divisão
2. ponto flutuante

##2. RV32IM -march=rv32im

Adiciona a extensão M:

1. multiplicação
2. divisão
3. resto

Novas instruções:

1. mul
2. mulh
3. div
4. rem

Necessário caso a microarquitetura implemente unidade multiplicadora/divisora.

##3. RV32IF -march=rv32if

Adiciona a extensão F:

1. ponto flutuante single precision (32 bits)
2. registradores f0-f31
3. operações IEEE754

Exemplos:

1. fadd.s
2. fmul.s
3. flw
4. fsw

Também é necessário utilizar ABI compatível: **-mabi=ilp32f**

Importante: FPU na microarquitetura $\neq$ uso automático de instruções floating-point

Mesmo que a microarquitetura implemente uma FPU, o compilador NÃO irá gerar instruções de ponto flutuante automaticamente se a ISA configurada for:

-march=rv32i ou **-march=rv32im**

Nesses casos:

1. o assembler rejeitará instruções floating-point;
2. o GCC gerará apenas código inteiro;
3. os registradores f0-f31 não serão utilizados.

Ou seja, a presença da FPU no hardware não é suficiente.

É obrigatório habilitar a extensão F explicitamente:

-march=rv32if -mabi=ilp32f

para que:

1. instruções floating-point sejam aceitas;
2. o compilador gere operações em ponto flutuante;
3. o Spike execute corretamente instruções da FPU.
4. Compatibilidade entre DUT, Spike e Toolchain

A ISA configurada deve ser consistente em todo o fluxo:

Componente	Configuração
Toolchain GCC/AS	-march=...
Spike	--isa=...
DUT	instruções implementadas
Parser/UVM	commits esperados

Exemplo para RV32IM:

LDFLAGS = -march=rv32im -mabi=ilp32 -nostdlib -T \$(LDFILE)

❏ 3. Configuração do Spike:

spike -isa=RV32IM

Exemplos de configuração:

1. Apenas ISA inteira básica **LDFLAGS = -march=rv32i -mabi=ilp32**

2. Com multiplicação/divisão **LDFLAGS = -march=rv32im -mabi=ilp32**
3. Com ponto flutuante **LDFLAGS = -march=rv32if -mabi=ilp32f**

Observação sobre incompatibilidades

Caso o programa utilize instruções não suportadas pela ISA configurada:

1. o assembler pode falhar;
2. o Spike pode gerar illegal instruction;
3. o DUT pode divergir do modelo de referência;
4. o scoreboard UVM irá detectar mismatch.

Portanto, sempre garanta que:

ISA do assembly = ISA compilada = ISA do Spike = ISA implementada no DUT

4. Simulação UVM

O Makefile utiliza o XCelium 2409 da Cadence. Para Quantus, VCS, etc, será necessário alterar o Makefile.

5. Premissas Arquiteturais

A microarquitetura implementada neste projeto segue uma abordagem **bare metal**, ou seja, sem sistema operacional ou mecanismo de gerenciamento de memória intermediário.

Por simplicidade didática, **não foi implementado decodificador de endereços** entre memória de instruções e memória de dados. Cada memória possui apenas a quantidade mínima de bits de endereço necessária para varrer internamente suas posições, assumindo portanto endereçamento local iniciado em 0x00000000.

Essa decisão é adequada para a proposta pedagógica do projeto, porém gera uma incompatibilidade com o simulador Spike, que executa programas em um espaço de memória unificado e espera endereços absolutos tipicamente iniciando em 0x80000000.

Para compatibilizar o modelo de referência com a microarquitetura **sem alterar o DUT**, foram adotadas as seguintes premissas:

△ 1. A microarquitetura não foi modificada

O DUT continua operando com endereçamento relativo local, exatamente como proposto originalmente.

Mesmo recebendo endereços completos de 32 bits, os módulos de memória utilizam apenas os bits menos significativos necessários para varrer suas posições internas.

△ 2. A adaptação foi realizada exclusivamente no `parser.py`

Por meio da variável:

`BASE = 0x80000000`

É definido o endereço base utilizado pelo **Spike**. Esse valor é utilizado para normalizar os *commits* gerados pelo Spike, realizando soma ou subtração quando necessário, de modo que:

- **Endereços absolutos do Spike** ($0x80000000 + \text{offset}$) sejam convertidos para endereços relativos esperados pelo **DUT** ($0x00000000 + \text{offset}$);
- **Valores de retorno de instruções** como `jal`, que escrevem `PC + 4` em registradores, também sejam ajustados para manter equivalência com a implementação *bare metal*.

Dessa forma, garante-se equivalência funcional entre o DUT e o modelo de referência, sem comprometer a simplicidade arquitetural originalmente proposta.

△ 3. Opção por não utilizar o pk (Proxy Kernel)

Durante os testes, também foi avaliada a utilização do pk (*Proxy Kernel*), frequentemente empregado junto ao Spike para execução de programas RISC-V em modo usuário. Entretanto, essa abordagem foi descartada neste projeto.

O pk funciona como uma camada intermediária semelhante a um sistema operacional simplificado, responsável por:

- Carregar o programa em memória;
- Alocar dinamicamente as seções `.text`, `.data` e `.bss`;
- Inicializar a pilha (*stack*);
- Tratar chamadas de sistema (`ecall`);
- Fornecer um ambiente de execução em *user mode*.

Como consequência, os endereços efetivos de código e dados deixam de ser fixos e passam a depender da política de alocação interna do pk, dificultando a definição de uma constante global BASE para normalização dos *commits*.

Em outras palavras, com o pk, o mapeamento de memória deixa de ser determinístico do ponto de vista do DUT, tornando a comparação direta com a microarquitetura *bare metal* significativamente mais complexa. Por esse motivo, optou-se por executar o Spike sem pk, permitindo controle explícito da região de memória utilizada pelo programa.

⚠ 4. O que seria necessário para suportar corretamente o pk

Para que a microarquitetura pudesse executar corretamente programas mediados pelo pk, seriam necessárias extensões arquiteturais adicionais, tais como:

- **Mapa de memória completo:** com decodificação explícita de regiões de ROM e RAM;
- **Suporte a endereçamento absoluto de 64 bits:** sem truncamento dos bits mais significativos;
- **Unificação ou arbitragem:** entre memória de instruções e memória de dados, aproximando-se de uma arquitetura menos estritamente Harvard;
- **Tratamento de exceções:** suporte mais completo a exceções e *traps*, especialmente relacionadas a *ecall*;
- **Modos privilegiados:** possibilidade futura de suporte a modos de execução privilegiados, caso se deseje executar software mais próximo de um ambiente real.

Essas extensões aumentariam a fidelidade arquitetural em relação a sistemas comerciais, porém adicionariam complexidade significativa ao projeto, fugindo do escopo didático originalmente proposto.